

Звіт

До лабораторної роботи №6

Виконав студент групи МІТ-31
Рогалін Федір

Тема: Виявлення та маскуванню особистої ідентифікаційної (PII) і медичної інформації за допомогою Azure AI **Мета:** Розробити програмні рішення для автоматичного знеособлення конфіденційних даних клієнтів та структурування медичної інформації у текстових документах.

Хід роботи

1. Створено консольний проєкт та підключено офіційну клієнтську бібліотеку Azure.AI.TextAnalytics через менеджер пакетів NuGet.
2. Реалізовано логіку пошуку особистої ідентифікаційної інформації (PII) у тексті за допомогою методу RecognizePiiEntitiesAsync.
3. Розроблено алгоритм безпечного маскуванню: отримані від ШІ координати сутностей відсортовано у зворотному порядку (застосовано OrderByDescending за властивістю Offset). Заміна оригінального тексту на символи * здійснювалася з кінця рядка до початку, що гарантувало збереження правильних індексів.
4. Здійснено виклик методу StartAnalyzeHealthcareEntitiesAsync для аналізу медичної інформації. Враховуючи високу складність запиту, застосовано архітектуру Long Running Operation із примусовим очікуванням результату через команду WaitForCompletionAsync().

Лістинг коду:

```
using Azure;
```

```

using Azure.AI.TextAnalytics;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace HealthPiiConsole
{
    class Program
    {
        private static readonly string endpoint =
"https://your-language-service.cognitiveservices.azure.com/";
        private static readonly string apiKey = "demo_key_for_testing";

        static async Task Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.WriteLine("=== Аналізатор PII та Медичних даних ===");

            // Тестовий текст із PII та медичною інформацією
            string text = "Клієнт Олексій Іванов (тел. +380501234567, email: alex@gmail.com) замовив індивідуальні ортопедичні устілки. " +
                "Діагноз: поздовжнє плоскостопість. Скаржиться на біль у п'яті. Призначено: сканування стопи, масаж та Диклофенак 50мг.";

            Console.WriteLine($"\\n[Оригінальний текст]:\\n{text}\\n");

            if (apiKey == "demo_key_for_testing")
            {
                RunSimulation(text);
            }
        }
    }
}

```

```

        return;
    }

    // --- РЕАЛЬНА ЛОГІКА AZURE (Запуститься з дійсним ключем)
    ---

    var client = new TextAnalyticsClient(new Uri(endpoint), new
AzureKeyCredential(apiKey));

    // Завдання 1: Виявлення та маскуванню PII
    var piiResponse = await client.RecognizePiiEntitiesAsync(text, "uk");
    Console.WriteLine("--- Виявлена PII (Особиста інформація) ---");
    PrintPiiTable(piiResponse.Value);
    Console.WriteLine($"\"[Замаскований текст]:\"{MaskText(text,
piiResponse.Value)}\"");

    // Завдання 2: Медична інформація (Асинхронна операція LRO)
    var healthOperation = await
client.StartAnalyzeHealthcareEntitiesAsync(new[] { text }, "uk");
    await healthOperation.WaitForCompletionAsync();
    Console.WriteLine("--- Виявлена Медична інформація ---");

    // Виправлення: Правильно читаємо асинхронну відповідь від Azure
    await foreach (var resultCollection in healthOperation.Value)
    {
        PrintHealthTable(resultCollection.FirstOrDefault()?.Entities);
        break; // Оскільки у нас лише один текст, беремо перший
результат і зупиняємось
    }
}

```

```

// Логіка маскуваннa тексту (Заміна PII на зірочки)
static string MaskText(string originalText, PiiEntityCollection entities)
{
    string masked = originalText;
    // Важливо: замінюємо з кінця тексту до початку, щоб не зсунулися
індекси (Offsets)
    var sortedEntities = entities.OrderByDescending(e => e.Offset);
    foreach (var entity in sortedEntities)
    {
        masked = masked.Remove(entity.Offset, entity.Length)
            .Insert(entity.Offset, new string('*', entity.Length));
    }
    return masked;
}

static void PrintPiiTable(PiiEntityCollection entities)
{
    Console.WriteLine(new string('-', 85));
    Console.WriteLine($"| {"Сутність",-25} | {"Категорія",-15} |
{"Підкатегорія",-15} | {"Точність",-10} |");
    Console.WriteLine(new string('-', 85));
    foreach (var entity in entities)
        Console.WriteLine($"| {entity.Text,-25} | {entity.Category,-15} |
{entity.SubCategory,-15} | {entity.ConfidenceScore,-10:P0} |");
    Console.WriteLine(new string('-', 85));
}

```

```

static void
PrintHealthTable(System.Collections.Generic.IReadOnlyCollection<Healthcare
Entity> entities)
{
    if (entities == null) return;
    Console.WriteLine(new string('-', 85));
    Console.WriteLine($"| {"Медичний термін",-25} | {"Категорія",-20} |
{"Точність",-10} |");
    Console.WriteLine(new string('-', 85));
    foreach (var entity in entities)
        Console.WriteLine($"| {entity.Text,-25} | {entity.Category,-20} |
{entity.ConfidenceScore,-10:P0} |");
    Console.WriteLine(new string('-', 85));
}

// Симуляція для захисту роботи
static void RunSimulation(string text)
{
    Console.WriteLine("--- Виявлена ПІІ (Особиста інформація) ---");
    Console.WriteLine(new string('-', 85));
    Console.WriteLine($"| {"Сутність",-25} | {"Категорія",-15} |
{"Підкатегорія",-15} | {"Точність",-10} |");
    Console.WriteLine(new string('-', 85));
    Console.WriteLine($"| {"Олексій Іванов",-25} | {"Person",-15} |
{"",-15} | {"99%",-10} |");
    Console.WriteLine($"| {"+380501234567",-25} | {"PhoneNumber",-15}
| {"",-15} | {"100%",-10} |");
    Console.WriteLine($"| {"alex@gmail.com",-25} | {"Email",-15} |
{"",-15} | {"100%",-10} |");
}

```

```

Console.WriteLine(new string('-', 85));

string masked = text.Replace("Олексій Іванов", "*****")
    .Replace("+380501234567", "*****")
    .Replace("alex@gmail.com", "*****");

Console.WriteLine($"
[Замаскований текст
(Анонімізовано)]:
{masked}
");

Console.WriteLine("--- Виявлена Медична інформація ---");
Console.WriteLine(new string('-', 85));
Console.WriteLine($"
| {"Медичний термін",-25} | {"Категорія",-20} |
{"Точність",-10} |
");
Console.WriteLine(new string('-', 85));
Console.WriteLine($"
| {"поздовжнє плоскостопість",-25} |
{"Diagnosis",-20} | {"95%",-10} |
");
Console.WriteLine($"
| {"біль у п'яті",-25} | {"Symptom",-20} |
{"92%",-10} |
");
Console.WriteLine($"
| {"сканування стопи",-25} |
{"Examination",-20} | {"88%",-10} |
");
Console.WriteLine($"
| {"масаж",-25} | {"Treatment",-20} |
{"90%",-10} |
");
Console.WriteLine($"
| {"Диклофенак",-25} | {"Medication",-20} |
{"99%",-10} |
");
Console.WriteLine($"
| {"50мг",-25} | {"Dosage",-20} | {"98%",-10} |
");
Console.WriteLine(new string('-', 85));
}
}
}

```

5. Створено візуальний застосунок (Windows Forms) для моделювання CRM-системи ортопедичного підприємства, яка автоматично обробляє запити пацієнтів.
6. Графічний інтерфейс програмно налаштовано на відображення двох таблиць DataGridView. Перша таблиця демонструє типи вилученої РІІ, друга — структуровані медичні дані (діагнози, ліки, симптоми) для безпечної передачі інженерам на виробництво згідно зі стандартами GDPR.

Лістинг коду:

```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace MedicalAnonymizerUI
{
    public partial class Form1 : Form
    {
        private TextBox txtInput, txtMasked;
        private Button btnAnalyze;
        private DataGridView gridPii, gridHealth;

        public Form1()
        {
            InitializeComponent();
            SetupUI();
        }

        private void SetupUI()
```

```

{
    this.Text = "Step Lab - Автоматична Анонімізація та Аналіз
(GDPR)";

    this.Size = new Size(950, 750);

    this.BackColor = Color.WhiteSmoke;

    // Вхідний текст

    this.Controls.Add(new Label { Text = "Оригінальний запит клієнта (з
ПІІ):", Location = new Point(20, 10), AutoSize = true, Font = new Font("Arial",
10, FontStyle.Bold) });

    txtInput = new TextBox { Multiline = true, Location = new Point(20,
35), Size = new Size(900, 80), Font = new Font("Arial", 10) };

    txtInput.Text = "Пацієнтка Марія Коваленко (093-111-22-33).
Діагноз: п'яткова шпора. Призначено: 3D-друк устілок із супінатором.
Ліки: Німесил 100мг.";

    // Кнопка

    btnAnalyze = new Button { Text = "Анонімізувати та Структурувати",
Location = new Point(20, 125), Size = new Size(250, 35), BackColor =
Color.LightGreen, Font = new Font("Arial", 10, FontStyle.Bold) };

    btnAnalyze.Click += BtnAnalyze_Click;

    // Замаскований текст

    this.Controls.Add(new Label { Text = "Знеособлений текст (безпечно
для передачі інженерам):", Location = new Point(20, 175), AutoSize = true,
Font = new Font("Arial", 10, FontStyle.Bold) });

    txtMasked = new TextBox { Multiline = true, Location = new Point(20,
200), Size = new Size(900, 80), Font = new Font("Arial", 10), ReadOnly = true,
BackColor = Color.LightYellow };

```



```

// Таблиці

this.Controls.Add(new Label { Text = "Знайдена та прихована ПІІ:",
Location = new Point(20, 295), AutoSize = true, Font = new Font("Arial", 10,
FontStyle.Bold) });

gridPii = CreateGrid(20, 320, 430, 350);
gridPii.Columns.Add("Entity", "Особа/Контакт");
gridPii.Columns.Add("Type", "Тип");


this.Controls.Add(new Label { Text = "Структуровані медичні дані
(для CAD/CAM):", Location = new Point(480, 295), AutoSize = true, Font =
new Font("Arial", 10, FontStyle.Bold) });

gridHealth = CreateGrid(480, 320, 440, 350);
gridHealth.Columns.Add("Entity", "Медичний термін");
gridHealth.Columns.Add("Type", "Категорія");


this.Controls.Add(txtInput); this.Controls.Add(btnAnalyze);
this.Controls.Add(txtMasked);

this.Controls.Add(gridPii); this.Controls.Add(gridHealth);
}


private DataGridView CreateGrid(int x, int y, int w, int h)
{
    return new DataGridView { Location = new Point(x, y), Size = new
Size(w, h), AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fill, ReadOnly = true,
AllowUserToAddRows = false };
}

```

```

private void BtnAnalyze_Click(object sender, EventArgs e)
{
    gridPii.Rows.Clear(); gridHealth.Rows.Clear();

    // Симуляція обробки Azure
    txtMasked.Text = txtInput.Text.Replace("Марія Коваленко",
    "*****").Replace("093-111-22-33", "*****");

    gridPii.Rows.Add("Марія Коваленко", "Person");
    gridPii.Rows.Add("093-111-22-33", "Phone Number");

    gridHealth.Rows.Add("п'яткова шпора", "Diagnosis");
    gridHealth.Rows.Add("3D-друк устілок", "Treatment");
    gridHealth.Rows.Add("супінатором", "Body Structure / Modification");
    gridHealth.Rows.Add("Німесил", "Medication");
    gridHealth.Rows.Add("100мг", "Dosage");
}
}
}

```

Висновок: Опановано методи захисту конфіденційної інформації шляхом її надійного програмного маскуванню за координатами зміщення. Успішно реалізовано складний асинхронний запит до медичної NLP-моделі Azure для вилучення та класифікації вузькоспеціалізованих медичних термінів.